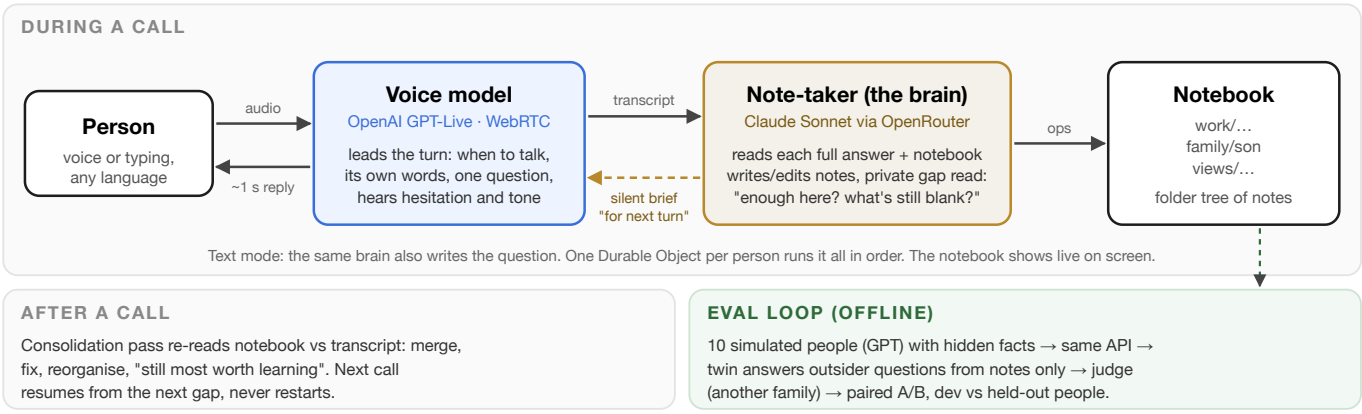
Viven Interviewer: design on one page

A voice-first interviewer that gets what makes a person *themselves* into notes a digital twin can answer from.

Utkarsh Kanwat · Nov 2026
viven-interviewer.utkarshkanwat.com

The question I designed for: how do you get tacit knowledge out of *any* human (an engineer, a nurse, a grandmother who cooks by feel), cheaply in their time, so they want to come back? **The metric:** how much of what a twin would need gets captured per minute, while they still enjoy it.



The voice model talks; the note-taker listens, writes, and quietly steers. Three model families, so none grades its own work.

DECISIONS AND WHY

1 The voice model leads; the brain only briefs it

The voice model hears tone and hesitation; my server only hears silence. So it decides when to speak and how, and the brain sends a silent "for your next turn" note. Reply time 2.4–7.8 s → 0.9–1.5 s; no more two questions spliced together. Cost: the brain can't control exact words. A late question is worse than a different one.

2 Model judgement over code rules

Every rule I coded (dig caps, repeat detector, trimming, filler regex) broke something human in testing: cut sentences, blocked a fair re-angle, cut off someone enjoying a thread. Now code only moves data and checks shape; behaviour is one shared purpose plus eight craft principles. The model gets *facts* about the turn, not orders.

3 Memory is a notebook, not a database or RAG

v1 stored typed items (story, value, rule…). The categories were mine, not theirs, and it stored the conversation ("dislikes being cut off") instead of the person. Now: a folder tree of short notes in their words, each folder described, rewritten only when seen in full. A few hours of talk fits in context, so retrieval would only add ways to miss things. Facts drawn out 0.12 → 0.23.

4 Only the delta, and what people will ask first

A base model already knows the typical answer, so only what makes *this* person different is worth their time. A real call went five questions deep on "how do you prioritise". It was faithful but useless: anyone would say that. So it now starts with what they're known for (what the twin's askers want), sketches the map of their life, and only goes deep where an answer is surprising.

5 A private "enough yet?" read on every answer

Standing rules lost to per-turn fields every time. So each answer gets a private read: is it complete, what would a fuller one add, and does this topic have their view plus one real case? Then move on. If the last questions all sat in one area, the next one goes elsewhere. Areas of life covered 2.0 → 2.8 per conversation, with no loss of facts (blind-graded).

6 The interface is part of the interview

Debug-looking screens make people perform. So: "What's your name?" → Start talking (or type). The notebook fills in live beside the conversation, so people *see* they're understood: the cheapest trust-builder I found. One End button; "call me later" by voice; coming back resumes from the next gap.

HOW I KNOW (AND WHAT DIDN'T WORK)

- **Real people first:** my own voice calls in English and Hinglish shaped decisions 1, 4 and 6. A competitor (TwinCafe) captures one expertise topic and saves a raw transcript; its twin hedges on anything personal.
- **Harness:** simulated people with hidden facts and "asks" an outsider would put to the twin; a judge from another model family; paired comparisons with confidence intervals; dev/held-out split so edits aren't fitted to the people I read; blind grading; cached judge calls.
- **Lost:** prompt rewrites ("go broad" made it dig *deeper*); quoted example words (it opened every line with "Right."); "one question mark" (it wrote "X, and Y?").
- **Won:** a few precise words on one thing at a time, checked by sampling the real call before and after.

STACK, AND WHAT'S NEXT

- TypeScript, one codebase: Node server *or* Cloudflare Workers + one Durable Object per person + D1 (free hosting).
- Vercel AI SDK via OpenRouter: retries, streaming, prompt caching (81% of input tokens cached; turn 5.4 s → 3.8 s).
- **Next:** ask "what would people ask this twin that the notes can't answer?" and chase the top gap; let people fix notes in place; stop a topic when its notes stop changing; the twin test on real people with their own questions.